

DSA Using C Programming - MCQ Questions (Paper Set-2)

::Q1::Which data structure allows insertion and deletion only at one end?{ =Stack ~Queue ~Array ~Graph } // Answer: Stack

::Q2::Which data structure allows insertion at the rear and deletion from the front?{ =Queue ~Stack ~Tree ~Heap } // Answer: Queue

::Q3::Which operation is performed when removing an element from a queue?{ =Delete (Dequeue) ~Insert (Enqueue) ~Peek ~Push } // Answer: Delete (Dequeue)

::Q4::Which operation is performed when inserting an element into a queue?{ =Insert (Enqueue) ~Delete (Dequeue) ~Peek ~Pop } // Answer: Insert (Enqueue)

::Q5::Which data structure is suitable for Breadth First Search (BFS)?{ =Queue ~Stack ~Linked List ~Tree } // Answer: Queue

::Q6::Which data structure is suitable for Depth First Search (DFS)?{ =Stack ~Queue ~Heap ~Array } // Answer: Stack

::Q7::What is the maximum number of children a binary tree node can have?{ =2 ~1 ~3 ~Unlimited } // Answer: 2

::Q8::Which searching algorithm checks every element one by one?{ =Linear Search ~Binary Search ~Hashing ~Tree Search } // Answer: Linear Search

::Q9::Which searching algorithm repeatedly divides the search space into two halves?{ =Binary Search ~Linear Search ~Bubble Search ~Interpolation Search } // Answer: Binary Search

::Q10::Which sorting algorithm has the best-case time complexity of $O(n)$?{ =Insertion Sort ~Selection Sort ~Bubble Sort ~Merge Sort } // Answer: Insertion Sort

::Q11::Which sorting algorithm always performs $O(n^2)$ comparisons regardless of input order?{ =Selection Sort ~Insertion Sort ~Merge Sort ~Quick Sort } // Answer: Selection Sort

::Q12::Which sorting algorithm is based on repeatedly swapping adjacent elements?{ =Bubble Sort ~Selection Sort ~Merge Sort ~Heap Sort } // Answer: Bubble Sort

::Q13::What is the last node of a linked list called?{ =Tail ~Head ~Root ~Top } // Answer: Tail

::Q14::Which linked list node contains both data and pointer?{ =Node ~Header ~Array ~Index } // Answer: Node

::Q15::Which operator is used to access structure members through a pointer?{ =-> ~. ~& ~* } // Answer: ->

::Q16::Which function allocates memory and initializes it to zero?{ =calloc() ~malloc() ~free() ~realloc() } // Answer: calloc()

::Q17::Which function changes the size of previously allocated memory?{ =realloc() ~malloc() ~calloc() ~free() } // Answer: realloc()

::Q18::Which notation represents the best-case complexity?{ =Big Omega (Ω) ~Big O ~Theta ~Sigma } // Answer: Big Omega (Ω)

::Q19::Which notation represents the average-case complexity?{ =Theta (Θ) ~Big O ~Big Omega ~Sigma } // Answer: Theta (Θ)

::Q20::Which notation represents the worst-case complexity?{ =Big O ~Theta ~Big Omega ~Sigma } // Answer: Big O

::Q21::Which data structure is used to implement function calls?{ =Stack ~Queue ~Array ~Heap } // Answer: Stack

::Q22::Which traversal visits every node level by level?{ =Level Order Traversal ~Preorder ~Inorder ~Postorder } // Answer: Level Order Traversal

::Q23::In a Binary Search Tree (BST), the left child contains values that are:{ =Less than the parent ~Greater than the parent ~Equal to the parent ~Random values } // Answer: Less than the parent

::Q24::In a Binary Search Tree (BST), the right child contains values that are:{ =Greater than the parent ~Less than the parent ~Equal to the parent ~Negative values } // Answer: Greater than the parent

::Q25::Which data structure is mainly used for scheduling processes in operating systems?{ =Priority Queue ~Stack ~Array ~Linked List } // Answer: Priority Queue

::Q26::Which of the following is a non-linear data structure?{ =Tree ~Array ~Stack ~Queue } // Answer: Tree

::Q27::Which of the following is a linear data structure?{ =Linked List ~Tree ~Graph ~Heap } // Answer: Linked List

::Q28::Which algorithm repeatedly selects the smallest element and places it at the beginning?{ =Selection Sort ~Bubble Sort ~Insertion Sort ~Quick Sort } // Answer: Selection Sort

::Q29::Which data structure uses nodes connected by pointers?{ =Linked List ~Array ~Stack ~Queue } // Answer: Linked List

::Q30::Which data structure is most suitable for implementing a browser's Back button?{ =Stack ~Queue ~Tree ~Graph } // Answer: Stack

SUNBEAM